SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q6 – A Directory of Names: Sorting and Searching Strings

```
#define LEN 30
char names[500][LEN];
for (int i = 0; i < n; i++) {
   if (!fgets(names[i], LEN, stdin)) break;
   names[i][strcspn(names[i], "\n")] = '\0';        /* remove the newline */
}
/* sort: insertion sort on names with strcmp and strcpy */
char tmp[LEN];
for (int i = 1; i < n; i++) {
   strcpy(tmp, names[i]); int j = i - 1;
   while (j >= 0 && strcmp(names[j], tmp) > 0) { strcpy(names[j+1], names[j]); j--; }
   strcpy(names[j+1], tmp);
}
/* binary search on names with strcmp; duplicates: names[i] and names[i+1] equal */
```

Student – Measurement and Improvement

Random names of 5–19 letters (about 13 bytes copied each), insertion sort on copies versus insertion sort on an index array (strcmp calls are the same):

n	strcmp calls	strcpy calls (copy sort)	Bytes copied (copy sort)	Index sort: int moves (4 bytes each)
100	2,486	2,587	32,262	2,389
500	61,289	61,795	810,177	60,797
5,000	6,239,453	6,244,457	81,405,096	6,234,459

Sorting indices does the same comparisons but replaces each 13-byte string copy (a function call plus a loop over the characters) with one 4-byte integer assignment – roughly 80 MB of copying avoided at n = 5,000.

Names of length 29 and 30 in names[][30]: fgets(names[i], 30, …) reads at most 29 characters plus the terminator. A 29-letter line leaves its newline unread, so the next fgets returns an empty name (measured: names[1] = "", length 0). A 30-letter line is truncated to 29 characters and its last letter and newline stay in the input, corrupting the next entry. So the longest storable name is 29 characters, and the input routine should read into a larger buffer (e.g. 64 bytes), check for the newline, and report too-long names instead of splitting them.

Binary search, n = 500: average 8.00 comparisons for present names and 9 for a missing name (⌊log2 500⌋ + 1 = 9).

Student – Design and Correctness

- **strcmp:** compares the two strings character by character as unsigned codes and returns 0 if equal, a negative value if the first is smaller at the first difference, and a positive value if larger. "apple" versus "Apple": the first letters are 'a' (97) and 'A' (65), so strcmp returns a positive number (measured: 1) and "Apple" sorts before "apple", with all upper-case names before all lower-case names.
- **Case-insensitive comparison:** compare tolower() of each pair of characters: int ci_cmp(const char *a, const char *b){ while (*a && *b) { int d = tolower((unsigned char)*a) − tolower((unsigned char)*b); if (d) return d; a++; b++; } return tolower((unsigned char)*a) − tolower((unsigned char)*b); } (ci_cmp("apple", "Apple") = 0 measured). strcasecmp is a POSIX alternative.
- **Cost of swapping two names:** a swap through a temporary is three strcpy calls, about 3 × (L + 1) bytes (up to 90 bytes for L = 29), against three integer assignments (12 bytes) for an index swap.
- **Duplicates:** equal strings compare equal, so after sorting they are adjacent; one pass comparing names[i] with names[i + 1] finds them in O(n). (For case-insensitive duplicates, sort with ci_cmp.)

Joint conclusion – why sort indices or pointers: each swap or shift moves 4–8 bytes instead of up to 30, with no copy loop. The trade-off is O(n) extra memory for the index array (2 KB for 500 names), an extra indirection (names[idx[j]]) on every comparison, and the names remain in their original positions, so every use of the sorted list goes through the index.

Alternative Concept and Complexity Analysis

```
char *list[500];
for (int i = 0; i < n; i++) list[i] = names[i];
static int cmp(const void *a, const void *b) { return strcmp(*(char * const *)a, *(char * const *)b); }
qsort(list, n, sizeof(char *), cmp);
```

qsort needs about n log2 n comparisons (measured 3,870 strcmp calls for n = 500, against 61,289 for the insertion sort on copied names), each costing up to L characters, so O(L · n log n) time. Each swap moves one 8-byte pointer instead of copying up to 30 bytes. Extra space: the pointer array of 500 × 8 = 4,000 bytes. As in the index sort, the names themselves never move.